

Platform Download Matrix

Fifteen download attempts driven through the app's own `/api/download` endpoint and Socket.IO event stream, exactly as the browser drives it. Six completed; nine did not, and most of those are fixable from our side rather than the platforms'.

6 COMPLETED	7 ERROR	2 INCONCLUSIVE	40% SUCCESS RATE	104MB RETRIEVED	15 PLATFORMS TESTED
FROM	TIME	STATUS	RETRIEVED	EXTRACTOR	OBSERVATION
Archive.org Keaton's Cops (1922), public domain	50.6 s	✓ COMPLETED	67.1 MB	ok · 2 fmt	Clean single-file <code>.ogv</code> . Slowest success, but bandwidth-bound rather than error-bound.
Pinterest yt-dlp regression-test pin	11.6 s	✓ COMPLETED	17.6 MB	ok · 6 fmt	A pre-merged MP4 was available, so the app's selector happened to match.
Twitch clip, yt-dlp regression test	9.4 s	✓ COMPLETED	15.5 MB	ok · 1 fmt	Clips serve one flat MP4. VODs and live streams were not tested.
Streamable yt-dlp regression-test clip	5.5 s	✓ COMPLETED	2.9 MB	ok · 1 fmt	Fastest video success of the run.
SoundCloud yt-dlp project test track	5.0 s	✓ COMPLETED	0.16 MB	ok · 2 fmt	Arrived already MP3-encoded, so it needed no ffmpeg. Audio requiring transcoding will still fail.
Instagram yt-dlp regression-test post	4.7 s	✓ COMPLETED	0.04 MB	fail	APP BUG Download worked and a valid H.264 MP4 landed, but <code>/api/info</code> crashes on this same URL, so the UI can never preview it.
YouTube Big Buck Bunny, official CC-BY upload	120.0 s	✗ ERROR	—	timeout	ENVIRONMENT + APP <code>Requested format is not available.</code> No JS runtime, so only HLS/DASH is exposed, and the app asks for a pre-merged file. Both halves proven fixable overleaf.
Reddit r/videos post	6.2 s	✗ ERROR	—	ok · 11 fmt	APP BUG Eleven formats found and the selector matched none of them. Reddit is DASH-only.
BiliBili yt-dlp regression-test video	6.2 s	✗ ERROR	—	ok · 11 fmt	APP BUG Identical to Reddit: extractor healthy, selector matched none of 11.
Vimeo from the repo's TEST_URLS.md	2.8 s	✗ ERROR	—	fail	ENVIRONMENT Extractor wants TLS impersonation and no target is available. One <code>pip install</code> away.
TikTok yt-dlp regression-test post	2.1 s	✗ ERROR	—	fail	ENVIRONMENT The same impersonation gap as Vimeo.
X / Twitter official Star Wars account	3.9 s	✗ ERROR	—	fail	PLATFORM HTTP error fetching the amplify card. Needs session cookies.
Facebook yt-dlp regression-test post	2.4 s	✗ ERROR	—	fail	PLATFORM <code>Cannot parse data.</code> Logged-out scraping is blocked; needs cookies.
Dailymotion yt-dlp regression-test video	1.7 s	~ INCONCLUSIVE	—	fail	DEAD TEST URL <code>Not found.</code> The video is gone upstream, so this row says nothing about Dailymotion.
YouTube yt-dlp project test video	120.0 s	~ INCONCLUSIVE	—	fail	DEAD TEST URL <code>Video unavailable.</code> Removed upstream. The slow failure is the missing JS runtime.

Reading the table. The *Extractor* column is the diagnostic: it reports whether `--dump-json` could read the page at all, with no format matching and no ffmpeg involved. Where the extractor succeeds but the download fails, the platform is fine and the fault is ours. Both 120 s rows hit the harness cap rather than finishing; the backend log holds their real terminal errors, quoted above. Without a JS runtime, YouTube extraction alone runs past two minutes. Times cover extraction plus transfer over one residential connection, so they are indicative rather than benchmarks.

Why the nine failed Grouped by root cause, largest first. Only two of the nine are the platform genuinely saying no.

ROOT CAUSE	BLOCKS	DETAIL	FIX
Format selector asks for a file that no longer exists	YouTube, Reddit, BiliBili — 3 platforms	<code>download.js</code> requests <code>best[height<=360]/best</code> , and <code>b</code> or <code>best</code> mean <i>one already-merged file</i> . Every major platform now ships video and audio as separate DASH streams, so the selector matches nothing while the extractor holds eleven good formats.	<code>bv*[height<=360]+ba/b[height<=360]/b</code> Proven on Reddit: resolves <code>dash-VIDEO-2+dash-AUDIO-1</code> . Merging needs ffmpeg , making it a hard prerequisite rather than an audio-only nicety.
No TLS impersonation backend	Vimeo, TikTok — 2 platforms	Both extractors try to impersonate a real browser's TLS fingerprint, find no target available, and abort before requesting a single byte. A missing optional dependency, not a block.	<code>python -m pip install curl_cffi</code> Cheapest win on the board: two platforms, one install, no code change.
No JavaScript runtime for YouTube	YouTube — headline platform	yt-dlp now needs a JS runtime to solve YouTube's player challenge and enables only <code>deno</code> by default. Without one, extraction is deprecated and degraded: formats go missing and each attempt crawls past two minutes. Node 24 is already installed; yt-dlp simply is not told to use it.	<code>--js-runtimes node</code> Verified: <code>JS runtimes: node-24.18.0</code> , then <code>SELECTED: 396+251 640x360 webm</code> . YouTube resolves once this and the selector fix are both in.
Genuinely requires a signed-in session	Facebook, X / Twitter — 2 platforms	These two are the platform actually refusing. Logged-out scraping is blocked, and no format selector or dependency changes that.	<code>--cookies-from-browser chrome</code> Handing your own session cookies to a server process is a real decision rather than a default. Worth discussing first.
Test URLs that died upstream	Dailymotion, 1 YouTube URL — 2 rows	These came from yt-dlp's own regression suite, so they were real URLs, but the content is gone. The rows measure nothing about the platforms.	Supply two current URLs and the harness re-runs in under a minute. Dailymotion in particular is likely a false negative.

Projected after the fixes. Installing ffmpeg and `curl_cffi`, correcting the selector and enabling the Node runtime addresses six of the nine failures: **6 → 11 passes, 40% → 73%**. Two remain cookie-gated and two need fresh URLs. YouTube, Reddit and BiliBili are proven; Vimeo and TikTok follow from a documented missing dependency. Treat 11 as the target to verify, not a promise.

App defects this run exposed Independent of platform coverage; these are ours to fix.

SEVERITY	DEFECT	DETAIL
BLOCKER	Nothing ever reaches the browser	All 104 MB landed in <code>downloads/</code> on the server and stayed there. No route serves that folder and no endpoint streams a file, so a web downloader cannot actually hand you the file. Every success in the table is invisible to a real user.
HIGH	<code>/api/info</code> crashes on multi-video posts	Instagram returned <code>Unexpected non-whitespace character after JSON at position 24016</code> . <code>--dump-json</code> emits one JSON object <i>per line</i> for multi-video posts, and <code>info.js</code> hands the whole buffer to a single <code>JSON.parse</code> . The download succeeded; only the preview broke.
HIGH	A warning is indistinguishable from a fatal error	<code>download.js</code> emits <code>download-error</code> on any stderr byte, and yt-dlp writes routine warnings there. On Vimeo, TikTok and YouTube the app raised <code>download-error</code> for mere <code>WARNING:</code> lines, flagged identically to a real failure. The harness had to ignore stderr and wait for a non-zero exit code to classify runs correctly. No success in this run happened to write to stderr, so no false failure was displayed, but format fallbacks and deprecation notices make that a matter of time.
MEDIUM	Path traversal in the delete endpoint	<code>download.js</code> builds the path with <code>path.join</code> before checking <code>startsWith</code> , but <code>join</code> has already collapsed the <code>../</code> segments, so the guard always passes and a crafted filename can delete outside <code>downloads/</code> .
LOW	Coalesced stdout chunks drop progress updates	Progress reporting otherwise works well: 114 percentage updates arrived, finely grained from 0.1% upward. But the parser takes only the <i>first</i> match per stdout chunk, and the log shows two progress lines arriving in one chunk, so updates are silently discarded when writes coalesce. Passing <code>--newline</code> and reading line by line fixes it.

SEVERITY	DEFECT	DETAIL
LOW	Filenames are not filesystem-safe	Saved files carry raw title characters, including fullwidth quotes in the Keaton file and stray symbols in the SoundCloud one. Add <code>--restrict-filenames</code> or <code>--windows-filenames</code> before this bites on a title containing a reserved character.

What this run does not tell you. One URL per platform, so a pass means that URL worked, not that the platform is solved: Twitch clips succeeded while VODs and live streams went untested. Playlists, age-restricted and geo-blocked content, and the app's audio-only path were all skipped, and audio cannot be assessed until ffmpeg is installed, since SoundCloud passed only because it was already MP3. Everything ran at 360p over one residential connection, so treat times as relative to each other. All fifteen test files were deleted from `downloads/` after measurement; raw JSON for both passes remains in the repo as `platform-test-results.json` and `info-probe-results.json`.